Using OP-TEE and QEMUv7

Tom Van Eyck

August 30, 2021

Contents

1 Prerequisites **3**
1.1 Ubuntu . . . 3
1.1.1 Changes from original [2] . . . 3
1.2 Google AOSP repo . . . 4
1.3 QEMU specific . . . 4

2 Building & running OP-TEE **5**
2.1 QEMU . . . 5
2.1.1 Download of repositories . . . 5
2.1.2 Toolchains . . . 5
2.1.3 Build OP-TEE . . . 5
2.1.4 Running OP-TEE . . . 5
2.2 Sabre Lite . . . 6

3 Debugging OP-TEE **6**
3.1 QEMU . . . 6
3.1.1 Download and install gdb for ARM . . . 6
3.1.2 Gdb settings . . . 6
3.1.3 Debugging OP-TEE . . . 7
3.2 Sabre Lite . . . 8

4 Custom Pseudo Trusted Application **8**
4.1 Example PTA . . . 8

5 Serial interrupt in secure world **13**
5.1 Interrupt handler . . . 13
5.2 Serial console driver . . . 14
5.2.1 Handling serial input . . . 15
5.3 CPU masks . . . 15

6 Bibliography **15**

1 Prerequisites

- Ubuntu 20.04 (only this distribution tested)
- Google AOSP repo
- QEMU

1.1 Ubuntu

This guide is based on a clean Ubuntu 20.04 installation on a Hyper-V VM. It should be possible to repeat the results on any Linux distribution and any hypervisor platform, as long as the required packages are available.

At least 30GiB free disk space is required. If the disk is not large enough and the build process is started, the disk will fill, possibly leading to a total system failure without chance of recovery.

The first thing to do when the operating system is installed, is to update the package manager for the right architectures. Run the following in a command line:

```
sudo dpkg --add-architecture i386
sudo apt update
```

Then the required packages can be installed:

```
sudo apt install android-tools-adb android-tools-fastboot autoconf \
        automake bc bison build-essential ccache cscope curl device-tree-compiler \
        expect flex ftp-upload gdisk iasl libattr1-dev libcap-dev \
        libfdt-dev libftdi-dev libglib2.0-dev libgmp-dev libhidapi-dev \
        libmpc-dev libncurses5-dev libncursesw5-dev libpixman-1-dev libpython2.7 \
        libssl-dev libtool make mtools netcat ninja-build python-crypto \
        python3-crypto python-is-python3 python-pyelftools python3-pycryptodome \
        python3-pyelftools python3-serial rsync unzip uuid-dev xdg-utils xterm \
        xz-utils zlib1g-dev
```

1.1.1 Changes from original [2]

- python-serial is not available for Ubuntu 20.04. This package has thus been removed from the install procedure. This should not have any impact, because only python3 is used.

- As python3 is the only python version available on Ubuntu 20.04 (as it should be), a package is added that links `python` to `python3`. This might be troublesome on systems where Python 2 is installed and existing tools use Python 2 by calling `python`. If this is the case, change these tools to use `python2` explicitly.

1.2 Google AOSP repo

The repo tool from the Google Android Open Source Project (AOSP) allows you to manage multiple git repositories using manifest files. OP-TEE uses these manifests to combine all projects into a single development environment based on the specific architecture to be used.

Obviously, because repo uses git, git needs to be installed and setup on the system. Configure git with your email and name.

```
sudo apt install git
git config --global user.email "<email>"
git config --global user.name "<name>"
```

After that, repo can be installed:

```
mkdir -p ~/bin
PATH="${HOME}/bin:${PATH}"
curl https://storage.googleapis.com/git-repo-downloads/repo > ~/bin/
    repo
chmod a+rx ~/bin/repo
```

Because repo is installed in `~/bin`, this directory needs to be added to the path. This should already been taken care of by Ubuntu, this can be checked if the following is present in `~/.profile`. If not, it should be added to the end of the file.

```
 set path so it includes user's private bin if it exists
if [ -d "$HOME/bin" ] ; then
    path="$HOME/bin:$PATH"
fi
```

1.3 QEMU specific

Next, QEMU should be installed. This can be done by executing `sudo apt install qemu`.

2 Building & running OP-TEE

2.1 QEMU

2.1.1 Download of repositories

OP-TEE uses a central repository containing the manifest files for repo for all architectures. To download all necessary git repositories of the OP-TEE project for the QEMUv7 architecture, execute the following commands:

```
mkdir -p <project-dir>
cd <project-dir>
repo init -u https://github.com/OP-TEE/manifest.git -m default.xml
repo sync -j4 --no-clone-bundle
```

2.1.2 Toolchains

The toolchains required to build any part of OP-TEE are included in one of the repositories downloaded in the previous step. To download the actual toolchains, execute

```
cd <project-dir>/build
make -j2 toolchains
```

2.1.3 Build OP-TEE

The OP-TEE project has three main parts: the operating system, the client API and the trusted applications. These three parts can be built on their own, but to get started it is easier to build the whole system at once. This is as simple as executing

```
cd <project-dir>/build
make -j `nproc`
```

This process will take some time.

2.1.4 Running OP-TEE

QEMU and OP-TEE can be launched by executing

```
cd <project-dir>/build
make run -j `nproc`

# After the command prompt (qemu) appears, press c to start execution.
```

```
c
```

This will launch QEMU in the current terminal and two new terminals, one serial connection to the secure world and one to the normal world. In the normal world terminal, you can login using the users `root`or `test`, after which it is possible to run programs in the normal world that interact with the secure world.

To test if everything went according to plan, run `xtest` once logged in. If there are no obvious errors, the build process should have completed successfully. You can run the example programs by executing any of the `optee_example_*` programs present in the path.

2.2 Sabre Lite

3 Debugging OP-TEE

To debug OP-TEE, some additional steps need to be taken to prepare everything.

3.1 QEMU

3.1.1 Download and install gdb for ARM

Normally, if this guide was followed and the toolchains were built accordingly, gdb should already be present in `<project-dir>/toolchains`. The necessary tools can be added to the path by adding the following in `.profile`.

```
if [ -d "<project-dir>/toolchains/aarch32/bin" ] && [ -d "<project-dir
    >/toolchains/aarch64/bin" ]; then
        PATH="<project-dir>/toolchains/aarch32/bin:<project-dir>/
            toolchains/aarch64/bin:$PATH"
fi
```

3.1.2 Gdb settings

To make the initialization of gdb easier, add the following at the end of `~/.gdbinit`:

```
set print pretty on

define optee
    handle SIGTRAP noprint nostop pass
    symbol-file <project-dir>/optee_os/out/arm/core/tee.elf
    target remote localhost:1234
```

```
end
document optee
    Loads and setup the binary (tee.elf) for OP-TEE and also connects to the QEMU
    remote.
end
```

3.1.3 Debugging OP-TEE

Because OP-TEE uses Address Space Layout Randomization (ASLR), it is necessary to disable it when starting QEMU. This is done by adding the flag `CFG_CORE_ASLR=n`.

Open two terminals. One will be the terminal where QEMU will be running, the other will be used to debug using gdb. Like before, execute the following with the added flag, but **do not press `c`**!

```
cd <project-dir>/build
make run CFG_CORE_ASLR=n -j `nproc`
```

Once QEMU is started, start gdb in the second terminal:

```
cd <project-dir>/toolchains/aarch32/bin
./arm-linux-gnueabihf-gdb -q
```

When the gdb prompt appears, run `optee` to load the symbols and to connect to QEMU. The output below should appear.

```
optee
> SIGTRAP is used by the debugger.
> Are you sure you want to change it? (y or n) [answered Y; input not from terminal]
> warning: No executable has been specified and target does not support
> determining executable automatically.  Try using the "file" command.
> 0x00000000 in ?? ()
```

Then it is possible to refer to the symbols in the OP-TEE p. To demonstrate, let's set a breakpoint on `tee_entry_std(...)`, an often called function and continue execution. Th

```
 Set breakpoint
b tee_entry_std
> Breakpoint 1 at 0xe1347ce: file core/tee/entry_std.c, line 528.

 Resume execution. Breakpoint should be hit.
c
> Continuing.
> [Switching to Thread 1.2]
>
> Thread 2 hit Breakpoint 1, tee_entry_std (arg=0xe300000, num_params=2) at core/tee/entry_std.c:528
```

```
> 528                 return __tee_entry_std(arg, num_params);
```

Now it is possible to further explore OP-TEE using gdb.

3.2 Sabre Lite

4 Custom Pseudo Trusted Application

Trusted Applications (TA) normally run in user mode in the Trusted Execution Environment. They are dynamically loaded when executed. When you want to run p in kernel mode, a Pseudo TA (PTA) is necessary. During compilation, PTA's are statically linked against the kernel.

It is possible to create a PTA by adding the sources to `optee_os/core/pta` and including them in the Makefile in the same directory. This way the PTA is included int the os when it is built.

It is then possible to create an application for the normal world that calls this PTA using its UUID and the right parameters, just like a normal TA. This application is created by copying an existing example application from `optee_examples` and removing the `ta` subfolder (as there will be no TA compiled separately). The Makefile of the host application should be updated with the right binary name and the build instructions for a TA should be removed from the parent (C)Makefiles.

4.1 Example PTA

This example shows a simple PTA that prints some text when called from the normal world.

```c
// File: optee_os/core/pta/testpta.c

#include <kernel/pseudo_ta.h>

#define PTA_NAME "test.pta"
#define PTA_TEST_PRINT_UUID { 0xd4be3f91, 0xc4e1, 0x436c, \
    { 0xb2, 0x92, 0xbf, 0xf5, 0x3e, 0x43, 0x04, 0xd5 } }

#define PTA_TEST_PRINT 0

static TEE_Result test_print(uint32_t type, TEE_Param p[TEE_NUM_PARAMS
    ]) {
    IMSG("This is a test of a pseudo trusted application.");

    return TEE_SUCCESS;
}

/*
 * Trusted Application Etnry Points
```

```
 */

static TEE_Result invoke_command(void *session __unused, uint32_t cmd,
                                 uint32_t ptypes,
                                 TEE_Param params[TEE_NUM_PARAMS]) {
  switch (cmd) {
    case PTA_TEST_PRINT:
        return test_print(ptypes, params);
        break;
    default:
        break;
  }

  return TEE_ERROR_NOT_IMPLEMENTED;
}

pseudo_ta_register(.uuid = PTA_TEST_PRINT_UUID, .name = PTA_NAME,
    .flags = PTA_DEFAULT_FLAGS, .invoke_command_entry_point =
        invoke_command);
```

```
# Added to optee_os/core/pta/sub.mk

srcs-y += testpta.c
```

The code for the host application:

```
// File: host/main.c

/*
 * Copyright (c) 2016, Linaro Limited
 * All rights reserved.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
    are met:
 *
 * 1. Redistributions of source code must retain the above copyright
    notice,
 * this list of conditions and the following disclaimer.
 *
 * 2. Redistributions in binary form must reproduce the above copyright
     notice,
 * this list of conditions and the following disclaimer in the
    documentation
 * and/or other materials provided with the distribution.
 *
 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
    "AS IS"
 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO
    , THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
```

```
 PURPOSE
 * ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE
 * LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
 * CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
 * SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
 * INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
 * CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
 * ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
 * POSSIBILITY OF SUCH DAMAGE.
 */

#include <err.h>
#include <stdio.h>
#include <string.h>

/* OP-TEE TEE client API (built by optee_client) */
#include <tee_client_api.h>

#define PTA_TEST_PRINT_UUID { 0xd4be3f91, 0xc4e1, 0x436c, \
    { 0xb2, 0x92, 0xbf, 0xf5, 0x3e, 0x43, 0x04, 0xd5 } }

#define PTA_TEST_PRINT 0

int main(void)
{
    TEEC_Result res;
    TEEC_Context ctx;
    TEEC_Session sess;
    TEEC_Operation op;
    TEEC_UUID uuid = PTA_TEST_PRINT_UUID;
    uint32_t err_origin;

    /* Initialize a context connecting us to the TEE */
    res = TEEC_InitializeContext(NULL, &ctx);
    if (res != TEEC_SUCCESS)
        errx(1, "TEEC_InitializeContext failed with code 0x%x", res);

    /*
     * Open a session to the "hello world" TA, the TA will print "hello
     * world!" in the log when the session is created.
     */
    res = TEEC_OpenSession(&ctx, &sess, &uuid,
                           TEEC_LOGIN_PUBLIC, NULL, NULL, &err_origin);
    if (res != TEEC_SUCCESS)
        errx(1, "TEEC_Opensession failed with code 0x%x origin 0x%x",
            res, err_origin);
```

```

    /*
     * Execute a function in the TA by invoking it, in this case
     * we're incrementing a number.
     *
     * The value of command ID part and how the parameters are
     * interpreted is part of the interface provided by the TA.
     */

    /* Clear the TEEC_Operation struct */
    memset(&op, 0, sizeof(op));

    /*
     * Prepare the argument. Pass a value in the first parameter,
     * the remaining three parameters are unused.
     */
    op.paramTypes = TEEC_PARAM_TYPES(TEEC_NONE, TEEC_NONE,
                     TEEC_NONE, TEEC_NONE);

    /*
     * TA_HELLO_WORLD_CMD_INC_VALUE is the actual function in the TA to be
     * called.
     */
    printf("Invoking PTA to print test.");
    res = TEEC_InvokeCommand(&sess, PTA_TEST_PRINT, &op, &err_origin);
    if (res != TEEC_SUCCESS)
        errx(1, "TEEC_InvokeCommand failed with code 0x%x origin 0x%x",
            res, err_origin);
    printf("Test printed.");

    /*
     * We're done with the TA, close the session and
     * destroy the context.
     *
     * The TA will print "Goodbye!" in the log when the
     * session is closed.
     */

    TEEC_CloseSession(&sess);

    TEEC_FinalizeContext(&ctx);

    return 0;
}
```

The Makefiles of the new host application:

```
# File: Makefile

export V?=0
```

```

# If _HOST or _TA specific compilers are not specified, then use CROSS_COMPILE
HOST_CROSS_COMPILE ?= $(CROSS_COMPILE)

.PHONY: all
all:
    $(MAKE) -C host CROSS_COMPILE="$(HOST_CROSS_COMPILE)" --no-builtin-variables

.PHONY: clean
clean:
    $(MAKE) -C host clean
```

```
# File: host/Makefile

CC      ?= $(CROSS_COMPILE)gcc
LD      ?= $(CROSS_COMPILE)ld
AR      ?= $(CROSS_COMPILE)ar
NM      ?= $(CROSS_COMPILE)nm
OBJCOPY ?= $(CROSS_COMPILE)objcopy
OBJDUMP ?= $(CROSS_COMPILE)objdump
READELF ?= $(CROSS_COMPILE)readelf

OBJS = main.o

CFLAGS += -Wall -I../ta/include -I$(TEEC_EXPORT)/include -I./include
#Add/link other required libraries here
LDADD += -lteec -L$(TEEC_EXPORT)/lib

BINARY = optee_example_testpta

.PHONY: all
all: $(BINARY)

$(BINARY): $(OBJS)
    $(CC) $(LDFLAGS) -o $@ $< $(LDADD)

.PHONY: clean
clean:
    rm -f $(OBJS) $(BINARY)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
```

```
# File: CMakeLists.txt

project (optee_example_testpta C)

set (SRC host/main.c)

```

```
add_executable (${PROJECT_NAME} ${SRC})

target_include_directories(${PROJECT_NAME}
            #    PRIVATE ta/include
               PRIVATE include)

target_link_libraries (${PROJECT_NAME} PRIVATE teec)

install (TARGETS ${PROJECT_NAME} DESTINATION ${CMAKE_INSTALL_BINDIR})
```

```
# File: Android.mk

# ~~~~~~~~~~~~~~~~~~~~~ optee-testpta ~~~~~~~~~~~~~~~~~~~~~
LOCAL_PATH := $(call my-dir)

include $(CLEAR_VARS)
LOCAL_CFLAGS += -DANDROID_BUILD
LOCAL_CFLAGS += -Wall

LOCAL_SRC_FILES += host/main.c

# LOCAL_C_INCLUDES := $(LOCAL_PATH)/ta/include

LOCAL_SHARED_LIBRARIES := libteec
LOCAL_MODULE := optee_example_testpta
LOCAL_VENDOR_MODULE := true
LOCAL_MODULE_TAGS := optional
include $(BUILD_EXECUTABLE)

# include $(LOCAL_PATH)/ta/Android.mk
```

5 Serial interrupt in secure world

To accept input on the serial console of the secure world, it is necessary to register an interrupt on that serial console. To do this, two things need to be done: a interrupt handler needs to be defined and the handler must be registered for the interrupt. The console interface might be different for QEMU or Sabre Lite, but the general way of working should be the same.

5.1 Interrupt handler

The interrupt handler is defined according to the `itr_handler` type in `<interrupt.h>`. This struct needs to be initialized with the interrupt location `it` (in this case `IT_CONSOLE_UART`), the flags for the interrupt (`ITRF_TRIGGER_LEVEL`) and the handler function. This function can accept a parameter

that is the `itr_handler` structure. When defining this struct, it is (probably) necessary to explicitly state that the interrupt handling is put in the right part of the binary. This is done by declaring `DECLARE_KEEP_PAGER(console_itr);`.

```
struct itr_handler {
    size_t it;
    uint32_t flags;
    itr_handler_t handler;
    void *data;
    SLIST_ENTRY(itr_handler) link;
};

static enum itr_return handler_function(struct itr_handler *h)
{
    // Handling code
    return ITRR_HANDLED;
}

static struct itr_handler handler = {
    .it = IT_CONSOLE_UART,
    .flags = ITRF_TRIGGER_LEVEL,
    .handler = handler_function,
};
DECLARE_KEEP_PAGER(handler);
```

When the interrupt handler is declared and defined, it can be used to enable the interrupt. First, the interrupt handler must be registered and then the interrupt can be enabled. Adding the interrupt handler is done by `itr_add(&handler)` and enabling the interrupt is done by `itr_enable(handler.it)`.

5.2 Serial console driver

To accept input from the serial console during the handling of the interrupt, it is necessary to have access to the console chip and driver. Because the driver is initialized during the boot sequence of the platform, the only way to get this access is by hijacking this process, disabling the standard interrupt enabled by the boot process and call our own function to save the serial console access location.

The original interrupt setup happens in the file `optee_os/core/arch/arm/plat-vexpress/main.c`. A macro called `driver_init(...)` is called with the function that enables the serial interrupt. This macro ensures that the interrupt enabling routine is called during the boot process. To prevent this from happening, the simplest solution is to comment out this line.

To save the access location of the console interface, the function `console_init(void)` in the same file calls a function `register_serial_console(...)` which is a wrapper for an assign operation.

We can imitate this function to do the same for us, by defining our own pointer to the serial chip and using this pointer to access the serial console.

```
static struct serial_chip *serial_chip;
```

5.2.1 Handling serial input

The serial console functions as follows: when data is sent to the serial console (for example by pressing a key in the console), it is accepted by the driver, the data is stored in a buffer and the cpu is notified by means of an interrupt. This interrupt invokes our interrupt handler function.

It is in this function that we are able to retrieve the data from the buffer. The interrupt keeps triggering if the buffer is not empty, thus it is necessary to empty the buffer after getting its contents. The following while loop gets a single character from the buffer and prints it as long as there are characters left in the buffer. It is here that we use the reference to the serial console.

```
static enum itr_return handler_function(struct itr_handler *h)
{
    while (serial_chip->ops->have_rx_data(serial_chip)) {
        int ch = serial_chip->ops->getchar(serial_chip);
        DMSG("new cpu %zu: got 0x%x", get_core_pos(), ch);
    }

    return ITRR_HANDLED;
}
```

5.3 CPU masks

Each system may have multiple cpus. When assigning an interrupt handler to a certain interrupt and enabling it, this interrupt is assigned to all cpus. This means that when an interrupt happens, both cpus will handle that interrupt, calling the same handler twice leading to undefined behavior.

To prevent this from happening, one can enable cpu masks on an interrupt to define which cpu is responsible for handling that interrupt. This is done by calling the function `itr_set_affinity(<interrupt>, <cpu_mask>)` with the interrupt from the handler and a cpu mask that is a sequence of bits, where a 1 at position n enables the handling of an interrupt by the cpu with number n.

It is not yet possible to handle interrupts on a cpu selected at interrupt handle time.

6 Bibliography

Based on multiple original sources from the OP-TEE team:

- [1] https://optee.readthedocs.io/en/latest/building/gits/build.html
- [2] https://optee.readthedocs.io/en/latest/building/prerequisites.html
- [3] https://github.com/ForgeRock/optee-build/blob/master/docs/debug.md#14-debug